

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME:Database Management Systems

COURSE CODE:CSA05

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 40%

QUESTIONS		
Q.No	Question	Bloom's Level
1	Given a set of relations and sample data, write SQL queries to retrieve specific information using joins and subqueries, and explain the logic used. Bloom's Level: BL3 – Apply	BL4 – Analyze
2	Using relational algebra, formulate expressions to solve complex queries such as retrieving records that satisfy multiple conditions across relations. Bloom's Level: BL4 – Analyse	BL5 – Evaluate
3	Given a real-world scenario (e.g., banking or student system), design a relational schema and construct appropriate SQL queries for data retrieval and updates. Bloom's Level: BL6 – Create	BL6 – Create
4	Analyse a given SQL query with nested subqueries and predict its output, explaining each step of execution. Bloom's Level: BL4 – Analyze	BL6 – Create
5	Compare different SQL approaches (joins vs subqueries) for solving the same problem and evaluate their efficiency. Bloom's Level: BL5 – Evaluate	BL4 – Analyze
6	Design and implement views for a database system to provide restricted data	BL5 –

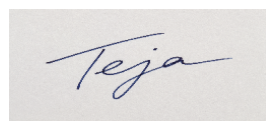
	access, and justify their use for security and abstraction. Bloom's Level: BL6 – Create	Evaluate
7	Given a database schema, write relational algebra expressions and convert them into equivalent SQL queries, analyzing differences in representation. Bloom's Level: BL4 – Analyze	BL4 – Analyze
8	Optimize a set of inefficient SQL queries by restructuring them using joins, indexing concepts, or views, and evaluate performance improvements. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate
9	Develop a complex SQL query involving grouping, aggregation, and nested queries to solve a real-world problem. Bloom's Level: BL6 – Create	BL6 – Create
10	Given multiple relations, analyze the query requirements and decide whether to use views, joins, or subqueries, justifying your choice with reasoning. Bloom's Level: BL5 – Evaluate	BL5 – Evaluate

Code of Conduct:

I TEJA VENKATA SWAROOP (192565098) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Q1. Retrieve specific information using SQL queries:

```
SELECT * FROM Student  
WHERE Marks > 80;
```

Using a Subquery:

```
SELECT Name, Marks  
FROM Student  
WHERE Marks >  
(  
SELECT AVG(Marks)  
FROM Student  
);
```

OUTPUT:

Logic:

- The subquery calculates the average marks.
- The outer query displays students whose marks are above the average.

```
mysql> SELECT * FROM Student
-> WHERE Marks > 80;
```

StudentID	Name	Department	Course	Age	Marks
101	Arun	CSE	DBMS	20	85
103	Rahul	CSE	DBMS	19	95
104	Sneha	EEE	Power Systems	22	88
105	Kiran	CSE	AI	20	91

4 rows in set (0.00 sec)

```
mysql> SELECT Name, Marks
-> FROM Student
-> WHERE Marks >
-> (
-> SELECT AVG(Marks)
-> FROM Student
-> );
```

Name	Marks
Rahul	95
Sneha	88
Kiran	91

3 rows in set (0.00 sec)

Q2. Relational Algebra

$$\pi_{\text{Name}}(\sigma_{\text{Department}='CSE' \wedge \text{Marks} > 80}(\text{Student}))$$

Equivalent SQL:

SELECT Name

FROM Student

WHERE Department='CSE'

AND Marks>80;

OUTPUT:

```
mysql> SELECT Name  
      -> FROM Student  
      -> WHERE Department='CSE'  
      -> AND Marks>80;
```

```
+-----+  
| Name  |  
+-----+  
| Arun  |  
| Rahul |  
| Kiran |  
+-----+
```

```
3 rows in set (0.00 sec)
```

```
mysql>
```

Q3. Real-world schema and SQL queries

Schema

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(30),  
    Department VARCHAR(20),
```

Course VARCHAR(30),
Age INT,
Marks INT
);

OUTPUT:

```
MySQL 8.0 Command Line - X + v

mysql> CREATE TABLE Student ( StudentID INT PRIMARY KEY, Name VARCHAR(30), Department VARCHAR(20), Course VARCHAR(30), Age INT, Marks INT );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO Student (StudentID, Name, Department, Course, Age, Marks)
-> VALUES
-> (101,'Arun','CSE','DBMS',20,85),
-> (102,'Priya','ECE','Networks',21,78),
-> (103,'Rahul','CSE','DBMS',19,95),
-> (104,'Sneha','EEE','Power Systems',22,88),
-> (105,'Kiran','CSE','AI',20,91);
Query OK, 5 rows affected (0.01 sec)
Records: 5 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM Student;
+-----+-----+-----+-----+-----+-----+
| StudentID | Name | Department | Course | Age | Marks |
+-----+-----+-----+-----+-----+-----+
| 101 | Arun | CSE | DBMS | 20 | 85 |
| 102 | Priya | ECE | Networks | 21 | 78 |
| 103 | Rahul | CSE | DBMS | 19 | 95 |
| 104 | Sneha | EEE | Power Systems | 22 | 88 |
| 105 | Kiran | CSE | AI | 20 | 91 |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> UPDATE Student SET Marks = 90 WHERE StudentID = 102;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT * FROM Student WHERE StudentID = 102;
+-----+-----+-----+-----+-----+-----+
| StudentID | Name | Department | Course | Age | Marks |
+-----+-----+-----+-----+-----+-----+
| 102 | Priya | ECE | Networks | 21 | 90 |
+-----+-----+-----+-----+-----+-----+
```

Q4. Analyse a Given SQL Query with Nested Subqueries and Predict Its Output

SQL Query

SELECT Name

```
FROM Student
WHERE Marks =
(
    SELECT MAX(Marks)
    FROM Student
);
```

Execute the Inner Query

```
SELECT MAX(Marks)
FROM Student;
```

OUTPUT:

```
mysql> SELECT Name FROM Student WHERE Marks = ( SELECT MAX(Marks) FROM Student );
+-----+
| Name |
+-----+
| Rahul |
+-----+
1 row in set (0.01 sec)

mysql> SELECT Name FROM Student WHERE Marks = 95;
+-----+
| Name |
+-----+
| Rahul |
+-----+
1 row in set (0.00 sec)

mysql>
```

Q5. Compare Different SQL Approaches (Joins vs Subqueries) for Solving the Same Problem and Evaluate Their Efficiency

Approach 1: Using a Subquery

SELECT StudentID, Name, Marks

FROM Student

WHERE Marks >

(

SELECT AVG(Marks)

FROM Student

);

OUTPUT:

```
MySQL 8.0 Command Line  X + v
+-----+
| Rahul |
+-----+
1 row in set (0.01 sec)

mysql> SELECT Name FROM Student WHERE Marks = 95;
+-----+
| Name |
+-----+
| Rahul |
+-----+
1 row in set (0.00 sec)

mysql> SELECT StudentID, Name, Marks FROM Student WHERE Marks > ( SELECT AVG(Marks) FROM Student );
+-----+-----+-----+
| StudentID | Name | Marks |
+-----+-----+-----+
| 102 | Priya | 90 |
| 103 | Rahul | 95 |
| 105 | Kiran | 91 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT StudentID, Name, Marks FROM Student WHERE Marks > 87.4;
+-----+-----+-----+
| StudentID | Name | Marks |
+-----+-----+-----+
| 102 | Priya | 90 |
| 103 | Rahul | 95 |
| 104 | Sneha | 88 |
| 105 | Kiran | 91 |
+-----+-----+-----+
4 rows in set (0.01 sec)

mysql>
```

Q6. Design and Implement Views for a Database System to Provide Restricted Data Access, and Justify Their Use

Create a View

The following view displays only the student's ID, Name, Department, and Marks while hiding other details such as Course and Age.

```
CREATE VIEW StudentView AS  
SELECT StudentID, Name, Department, Marks  
FROM Student;
```

Display the View

```
SELECT * FROM StudentView;
```

OUTPUT:

```
mysql> CREATE VIEW StudentView AS SELECT StudentID, Name, Department, Marks FROM Student;  
Query OK, 0 rows affected (0.01 sec)  
  
mysql> ^C  
mysql> SELECT * FROM StudentView;  
+-----+-----+-----+-----+  
| StudentID | Name | Department | Marks |  
+-----+-----+-----+-----+  
|      101 | Arun | CSE        | 85    |  
|      102 | Priya | ECE        | 90    |  
|      103 | Rahul | CSE        | 95    |  
|      104 | Sneha | EEE        | 88    |  
|      105 | Kiran | CSE        | 91    |  
+-----+-----+-----+-----+  
5 rows in set (0.00 sec)
```

Q7. Given a Database Schema, Write Relational Algebra Expressions and Convert Them into Equivalent SQL Queries, Analyzing Differences in Representation

Relational Algebra Expression

```
[  
  \pi_{\{Name\}}(\sigma_{\{Marks>90\}}(Student))\right)
```

Explanation:

- σ (Selection): Selects students whose Marks > 90.
- π (Projection): Displays only the Name column.

Equivalent SQL Query

```
SELECT Name  
FROM Student  
WHERE Marks > 90;
```

OUTPUT:

```
MySQL 8.0 Command Line - X + -  
+-----+  
| StudentID | Name | Marks |  
+-----+  
| 102 | Priya | 90 |  
| 103 | Rahul | 95 |  
| 104 | Sneha | 88 |  
| 105 | Kiran | 91 |  
+-----+  
4 rows in set (0.01 sec)  
  
mysql> CREATE VIEW StudentView AS SELECT StudentID, Name, Department, Marks FROM Student;  
Query OK, 0 rows affected (0.01 sec)  
  
mysql> ^C  
mysql> SELECT * FROM StudentView;  
+-----+  
| StudentID | Name | Department | Marks |  
+-----+  
| 101 | Arun | CSE | 85 |  
| 102 | Priya | ECE | 90 |  
| 103 | Rahul | CSE | 95 |  
| 104 | Sneha | EEE | 88 |  
| 105 | Kiran | CSE | 91 |  
+-----+  
5 rows in set (0.00 sec)  
  
mysql> SELECT Name FROM Student WHERE Marks > 90;  
+-----+  
| Name |  
+-----+  
| Rahul |  
| Kiran |  
+-----+  
2 rows in set (0.00 sec)  
  
mysql> |
```

Q8. Optimize a Set of Inefficient SQL Queries by Restructuring Them Using Indexing Concepts or Views, and Evaluate Performance Improvements

Inefficient Query

The following query retrieves all columns, even though only the student's name and marks are required.

```
SELECT *  
FROM Student  
WHERE Marks > 80;
```

Optimized Query

Retrieve only the required columns.

```
SELECT Name, Marks  
FROM Student  
WHERE Marks > 80;
```

OUTPUT:

```
mysql> SELECT * FROM Student WHERE Marks > 80;  
+-----+-----+-----+-----+-----+-----+  
| StudentID | Name | Department | Course | Age | Marks |  
+-----+-----+-----+-----+-----+-----+  
| 101 | Arun | CSE | DBMS | 20 | 85 |  
| 102 | Priya | ECE | Networks | 21 | 90 |  
| 103 | Rahul | CSE | DBMS | 19 | 95 |  
| 104 | Sneha | EEE | Power Systems | 22 | 88 |  
| 105 | Kiran | CSE | AI | 20 | 91 |  
+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.00 sec)  
  
mysql> SELECT Name, Marks FROM Student WHERE Marks > 80;  
+-----+-----+  
| Name | Marks |  
+-----+-----+  
| Arun | 85 |  
| Priya | 90 |  
| Rahul | 95 |  
| Sneha | 88 |  
| Kiran | 91 |  
+-----+-----+  
5 rows in set (0.00 sec)  
  
mysql> |
```

Q9. Develop a Complex SQL Query Involving Grouping, Aggregation, and Nested Queries to Solve a Real-World Problem

SQL Query

S

SELECT Department, AVG(Marks) AS AverageMarks

FROM Student

GROUP BY Department

HAVING AVG(Marks) >

(

SELECT AVG(Marks)

FROM Student

);

SELECT AVG(Marks)

FROM Student;

OUTPUT:

```
mysql> SELECT Department, AVG(Marks) AS AverageMarks FROM Student GROUP BY Department HAVING AVG(Marks) > ( SELECT AVG(Marks) FROM Student );
+-----+-----+
| Department | AverageMarks |
+-----+-----+
| CSE       | 90.3333      |
| ECE       | 90.0000      |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT AVG(Marks) FROM Student;
+-----+
| AVG(Marks) |
+-----+
| 89.8000    |
+-----+
1 row in set (0.00 sec)

mysql> |
```

Q10. Given Multiple Query Requirements, Decide Whether to Use Views, Joins, or Subqueries and Justify Your Choice

Requirement 1: Display only Student ID, Name, Department, and Marks

CREATE VIEW StudentView AS

SELECT StudentID, Name, Department, Marks

FROM Student;

Requirement 2: Find Students Who Scored Above the Average Marks

Subquery

SELECT Name, Marks

FROM Student

WHERE Marks >

(

SELECT AVG(Marks)

FROM Student

);

Requirement 3: Combine Data from Multiple Tables

SELECT S.Name, D.DepartmentName

FROM Student S

JOIN Department D

ON S.DepartmentID = D.DepartmentID;

OUTPUT:

```
mysql> SELECT Name, Marks FROM Student WHERE Marks > ( SELECT AVG(Marks) FROM Student );
```

Name	Marks
Priya	90
Rahul	95
Kiran	91

3 rows in set (0.00 sec)